

Generics

1. Implement the hands-on 1 using Sortedlist being EmployeeID as a key

```
using System;
using System.Collections;
class Employee
{
    public string EmployeeName;
    public int EmployeeID;
    public double Salary;
    public Employee(string employeeName, int employeeID, double salary)
    {
        EmployeeName = employeeName;
        EmployeeID = employeeID;
        Salary = salary;
    }
}
class EmployeeDAL
{
    SortedList employees = new SortedList();
    public bool AddEmployee(Employee e)
    {
        employees.Add(e.EmployeeID, e);
        return true;
    }
    public bool DeleteEmployee(int id)
    {

```

```

        if (employees.ContainsKey(id))
        {
            employees.Remove(id);

            return true;
        }

        return false;
    }

    public string SearchEmployee(int id)
    {
        if (employees.ContainsKey(id))
        {
            Employee e = (Employee)employees[id];

            return e.EmployeeName;
        }

        return null;
    }

    public Employee[] GetAllEmployeesListAll()
    {
        Employee[] allEmployees = new Employee[employees.Count];

        for (int i = 0; i < employees.Count; i++)
        {
            allEmployees[i] = (Employee)employees.GetByIndex(i);
        }

        return allEmployees;
    }
}

```

```
class Program
{
    static void Main()
    {
        EmployeeDAL dal = new EmployeeDAL();
        Employee e1 = new Employee("Pooja", 103, 55000);
        Employee e2 = new Employee("Rahul", 101, 60000);
        Employee e3 = new Employee("Anu", 102, 50000);
        dal.AddEmployee(e1);
        dal.AddEmployee(e2);
        dal.AddEmployee(e3);
        Console.WriteLine("Employees added successfully.");
        string name = dal.SearchEmployee(102);
        if (name != null)
        {
            Console.WriteLine("Employee found: " + name);
        }
        else
        {
            Console.WriteLine("Employee not found.");
        }
        bool deleted = dal.DeleteEmployee(103);
        if (deleted)
        {
            Console.WriteLine("Employee deleted successfully.");
        }
        else
        {

```

```

        Console.WriteLine("Employee not found.");
    }

    Console.WriteLine();

    Console.WriteLine("All Employees:");

    Employee[] allEmployees = dal.GetAllEmployeesListAll();

    foreach (Employee e in allEmployees)
    {
        Console.WriteLine(
            "ID: " + e.EmployeeID +
            ", Name: " + e.EmployeeName +
            ", Salary: " + e.Salary
        );
    }
}

```

OUTPUT:

Employees added successfully.

Employee found: Anu

Employee deleted successfully.

All Employees:

ID: 101, Name: Rahul, Salary: 60000

ID: 102, Name: Anu, Salary: 50000

2.Modify hands-on 1 with Generic List

```
using System;
```

```
using System.Collections.Generic;
```

```
class Employee
{
    public string EmployeeName;
    public int EmployeeID;
    public double Salary;

    public Employee(string employeeName, int employeeID, double salary)
    {
        EmployeeName = employeeName;
        EmployeeID = employeeID;
        Salary = salary;
    }
}
```

```
class EmployeeDAL
{
    List<Employee> employees = new List<Employee>();

    public bool AddEmployee(Employee e)
    {
        employees.Add(e);
        return true;
    }

    public bool DeleteEmployee(int id)
    {
        for (int i = 0; i < employees.Count; i++)
```

```
{  
    if (employees[i].EmployeeID == id)  
    {  
        employees.RemoveAt(i);  
        return true;  
    }  
}  
  
return false;  
}
```

```
public string SearchEmployee(int id)  
{  
    foreach (Employee e in employees)  
    {  
        if (e.EmployeeID == id)  
        {  
            return e.EmployeeName;  
        }  
    }  
  
    return null;  
}
```

```
public Employee[] GetAllEmployeesListAll()  
{  
    return employees.ToArray();  
}
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        EmployeeDAL dal = new EmployeeDAL();
```

```
        dal.AddEmployee(new Employee("Pooja", 101, 50000));
```

```
        dal.AddEmployee(new Employee("Rahul", 102, 60000));
```

```
        dal.AddEmployee(new Employee("Anu", 103, 55000));
```

```
        Console.WriteLine("Employees added successfully.");
```

```
        string name = dal.SearchEmployee(102);
```

```
        if (name != null)
```

```
        {
```

```
            Console.WriteLine("Employee found: " + name);
```

```
        }
```

```
        else
```

```
        {
```

```
            Console.WriteLine("Employee not found.");
```

```
        }
```

```
        bool deleted = dal.DeleteEmployee(103);
```

```
        if (deleted)
```

```
{  
    Console.WriteLine("Employee deleted successfully.");  
}  
  
else  
  
{  
    Console.WriteLine("Employee not found.");  
}  
  
  
Console.WriteLine();  
Console.WriteLine("All Employees:");  
  
  
Employee[] allEmployees = dal.GetAllEmployeesListAll();  
  
foreach (Employee e in allEmployees)  
{  
    Console.WriteLine(  
        "ID: " + e.EmployeeID +  
        ", Name: " + e.EmployeeName +  
        ", Salary: " + e.Salary  
    );  
}  
}
```

OUTPUT:

Employees added successfully.

Employee found: Rahul

Employee deleted successfully.

All Employees:

ID: 101, Name: Pooja, Salary: 50000

ID: 102, Name: Rahul, Salary: 60000

3.Create a Program class to accept a string, containing multiple words with alphabets and digits. Each alphabet should be added to the AlphaList and each digit should be added to the DigitList. And sort both the lists.

```
using System;
```

```
using System.Collections.Generic;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        List<char> AlphaList = new List<char>();
```

```
        List<char> DigitList = new List<char>();
```

```
        Console.Write("Enter a string: ");
```

```
        string str = Console.ReadLine();
```

```
        foreach (char ch in str)
```

```
        {
```

```
            if (char.IsLetter(ch))
```

```
            {
```

```
                AlphaList.Add(ch);
```

```
            }
```

```

        else if (char.IsDigit(ch))
        {
            DigitList.Add(ch);
        }
    }

    AlphaList.Sort();
    DigitList.Sort();

    Console.WriteLine("AlphaList: " + string.Join(" ", AlphaList));
    Console.WriteLine("DigitList: " + string.Join(" ", DigitList));
}
}

```

OUTPUT:

Enter a string: hello345

AlphaList: e, h, l, l, o

DigitList: 3, 4, 5

4) Create an Employee class with the following auto implemented properties.

- 1. EmployeeID as integer**
- 2. EmployeeName as string**
- 3. Designation as string**
- 4. JoiningDate as DateTime**
- 5. Department Name as string**

Create another class with the name, EmployeeData having the auto implemented properties,

- 1. EmployeeInfo as List**

**Define a method that reads the employee details and stores them in a file in Csv format.
To do this, the method should first read and add the employee details to the EmployeeInfo**

collection. Later the details present in the list should be stored in a file. Ensure that the previous data has to be retained everytime a new data is added.

```
using System;

using System.Collections.Generic;

using System.IO;

class Employee
{
    public int EmployeeID { get; set; }

    public string EmployeeName { get; set; }

    public string Designation { get; set; }

    public DateTime JoiningDate { get; set; }

    public string DepartmentName { get; set; }
}

class EmployeeData
{
    public List<Employee> EmployeeInfo { get; set; }

    public EmployeeData()
    {
        EmployeeInfo = new List<Employee>();
    }

    public void AddEmployee()
    {
        Employee emp = new Employee();

        Console.Write("Enter Employee ID: ");

        emp.EmployeeID = Convert.ToInt32(Console.ReadLine());

        Console.Write("Enter Employee Name: ");

        emp.EmployeeName = Console.ReadLine();

        Console.Write("Enter Designation: ");
```

```

emp.Designation = Console.ReadLine();
Console.Write("Enter Joining Date (dd/MM/yyyy): ");
emp.JoiningDate = Convert.ToDateTime(Console.ReadLine());
Console.Write("Enter Department Name: ");
emp.DepartmentName = Console.ReadLine();
EmployeeInfo.Add(emp);
SaveToCSV();
}

public void SaveToCSV()
{
    string fileName = "Employee.csv";
    bool fileExists = File.Exists(fileName);
    using (StreamWriter sw = new StreamWriter(fileName, true))
    {
        if (!fileExists)
        {
            sw.WriteLine("EmployeeID,EmployeeName,Designation,JoiningDate,DepartmentName");
        }
        foreach (Employee emp in EmployeeInfo)
        {
            sw.WriteLine(emp.EmployeeID + "," +
                emp.EmployeeName + "," +
                emp.Designation + "," +
                emp.JoiningDate.ToString("dd/MM/yyyy") + "," +
                emp.DepartmentName);
        }
    }
    Console.WriteLine("\nEmployee details saved successfully.");
}

```

```

    }
}
class Program
{
    static void Main()
    {
        EmployeeData data = new EmployeeData();
        Console.Write("Enter number of employees: ");
        int n = Convert.ToInt32(Console.ReadLine());
        for (int i = 0; i < n; i++)
        {
            Console.WriteLine("\nEnter Employee Details " + (i + 1));
            data.AddEmployee();
        }
        Console.WriteLine("\nCSV File Created and Data Stored Successfully.");
        Console.ReadLine();
    }
}

```

OUTPUT:

Enter Employee Name: ram

Enter Designation: sales

Enter Joining Date (dd/MM/yyyy): 12/10/2020

Enter Department Name: cse

Employee details saved successfully.

Enter Employee Details 2

Enter Employee ID: 2

Enter Employee Name: hari

Enter Designation: tester

Enter Joining Date (dd/MM/yyyy): 14/09/2005

Enter Department Name: sales

Employee details saved successfully.

CSV File Created and Data Stored Successfully.

5) Create a console application to manage a queue of customer service requests in banking domain. The application has to create a service token and manage it. The application must have a class, "ServiceToken" with the auto implemented properties TokenID as integer (auto generated), Position, TicketDateTime as DateTime, Status as string. This class must have a instance constructor to assign the values to these properties.

The application should define another class, TicketManager, with Queue as an auto implemented property. In this class define the below methods with the mentioned behavior.

- 1. GenerateServiceToken():** This method should create an instance of ServiceToken class and add to the List.
- 2. GetNextToken():** This method should get the next available token from the Queue.
- 3. UpdateToken(int TokenID):** This method should update the corresponding token's status to complete.
- 4. SkipToken():** This method should skip the immediate next token and return the subsequent token.

Write a driver code to display the UI as below and call the respective operations when an option is selected.

TOKEN MANAGEMENT SYSTEM

- 1. Create Token**
- 2. Get Next Token**
- 3. Update Token**
- 4. Skip Token**
- 5. List all tokens**
- 6. Exit**

Enter your Choice?

If option 1 is selected, call GenerateServiceToken() method,

If option 2 is selected, call GetNextToken(),

If option 3 is selected, call the UpdateToken(int TokenID) by passing the current token ID

If option 4 is selected, call SkipToken(),

If option 5 is selected, display all the tokens information,

If option 6 is selected, exit the application

using System;

using System.Collections.Generic;

class ServiceToken

```
{  
    public int TokenID { get; set; }  
    public int Position { get; set; }  
    public DateTime TicketDateTime { get; set; }  
    public string Status { get; set; }  
    public ServiceToken(int tokenID, int position, DateTime dateTime, string status)  
    {  
        TokenID = tokenID;  
        Position = position;  
        TicketDateTime = dateTime;  
        Status = status;  
    }  
}
```

class TicketManager

```
{  
    public Queue<ServiceToken> Queue { get; set; }  
    int tokenCount = 1;  
    public TicketManager()  
    {
```

```

        Queue = new Queue<ServiceToken>();
    }

    public void GenerateServiceToken()
    {
        ServiceToken token = new ServiceToken(
            tokenCount,
            Queue.Count + 1,
            DateTime.Now,
            "Waiting"
        );
        Queue.Enqueue(token);
        Console.WriteLine("Token Generated Successfully.");
        Console.WriteLine("Token ID : " + token.TokenID);
        tokenCount++;
    }

    public ServiceToken GetNextToken()
    {
        if (Queue.Count > 0)
        {
            ServiceToken token = Queue.Peek();
            Console.WriteLine("Next Token ID : " + token.TokenID);
            return token;
        }
        Console.WriteLine("No Tokens Available.");
        return null;
    }

    public void UpdateToken(int TokenID)
    {

```



```

foreach (ServiceToken token in Queue)
{
    if (token.TokenID == TokenID)
    {
        token.Status = "Completed";

        Console.WriteLine("Token Updated Successfully.");

        return;
    }
}

Console.WriteLine("Token Not Found.");
}

public ServiceToken SkipToken()
{
    if (Queue.Count > 1)
    {
        Queue.Dequeue();

        ServiceToken token = Queue.Peek();

        Console.WriteLine("Skipped Token.");

        Console.WriteLine("Next Token ID : " + token.TokenID);

        return token;
    }

    Console.WriteLine("No Token Available to Skip.");

    return null;
}

public void DisplayTokens()
{
    if (Queue.Count == 0)
    {

```

```

        Console.WriteLine("No Tokens Available.");
        return;
    }
    Console.WriteLine("\nToken Details");
    foreach (ServiceToken token in Queue)
    {
        Console.WriteLine("-----");
        Console.WriteLine("Token ID    : " + token.TokenID);
        Console.WriteLine("Position   : " + token.Position);
        Console.WriteLine("Ticket Time : " + token.TicketDateTime);
        Console.WriteLine("Status     : " + token.Status);
    }
}

class Program
{
    static void Main()
    {
        TicketManager manager = new TicketManager();
        int choice;
        do
        {
            Console.WriteLine("\nTOKEN MANAGEMENT SYSTEM");
            Console.WriteLine("-----");
            Console.WriteLine("1. Create Token");
            Console.WriteLine("2. Get Next Token");
            Console.WriteLine("3. Update Token");
            Console.WriteLine("4. Skip Token");

```

```
Console.WriteLine("5. List all tokens");
Console.WriteLine("6. Exit");
Console.Write("\nEnter your Choice: ");
choice = Convert.ToInt32(Console.ReadLine());
switch(choice)
{
    case 1:
        manager.GenerateServiceToken();
        break;
    case 2:
        manager.GetNextToken();
        break;
    case 3:
        Console.Write("Enter Token ID: ");
        int id = Convert.ToInt32(Console.ReadLine());
        manager.UpdateToken(id);
        break;
    case 4:
        manager.SkipToken();
        break;
    case 5:
        manager.DisplayTokens();
        break;
    case 6:
        Console.WriteLine("Exiting Application...");
        break;
    default:
        Console.WriteLine("Invalid Choice.");
```

```
        break;
    }
} while(choice != 6);
Console.ReadLine();
}
}
```

OUTPUT:

TOKEN MANAGEMENT SYSTEM

1. Create Token
2. Get Next Token
3. Update Token
4. Skip Token
5. List all tokens
6. Exit

Enter your Choice: 1

Token Generated Successfully.

Token ID : 1

TOKEN MANAGEMENT SYSTEM

1. Create Token
2. Get Next Token
3. Update Token
4. Skip Token
5. List all tokens
6. Exit

Enter your Choice: 2

Next Token ID : 1

TOKEN MANAGEMENT SYSTEM

1. Create Token
2. Get Next Token
3. Update Token
4. Skip Token
5. List all tokens
6. Exit

Enter your Choice: 3

Enter Token ID: 1

Token Updated Successfully.

TOKEN MANAGEMENT SYSTEM

1. Create Token
2. Get Next Token
3. Update Token
4. Skip Token
5. List all tokens
6. Exit

Enter your Choice: 5

Token Details

Token ID : 1

Position : 1

Ticket Time : 25-07-2026 09:15:49

Status : Completed

TOKEN MANAGEMENT SYSTEM

1. Create Token

2. Get Next Token

3. Update Token

4. Skip Token

5. List all tokens

6. Exit

Enter your Choice: 6

Exiting Application...